Learning Objectives

Learners will be able to...

- **Create a basic reservation system for a restaurant**
- **Add project to student GitHub portfolio**

Project Intro

Restaurant Management System Project

Welcome to your next project in the Data Structures Course! In this assignment, you will be developing a simple restaurant management system using C++. This system will help you understand and apply various data structure concepts such as lists and maps. By the end of this project, you will have created a system that can manage a restaurant's waitlist and table layout.

Project Overview

In this project, you will create a C++ program that can:

- Add parties to a waitlist with their name, party size, and contact information.
- Manage the restaurant's table layout, including the number of seats at each table and whether they are occupied.
- Seat parties from the waitlist at available tables based on their size.
- Clear tables when parties leave, making them available for new parties.
- View the current waitlist and the status of each table in the restaurant.

Key Concepts We Will Apply

Maps: You will use maps to manage the restaurant layout, enabling efficient lookup, addition, and update of table statuses. This will be crucial for functions like seating a party at a table or updating a table's availability.

Lists: The waitlist management will utilize lists, allowing you to dynamically add parties to the waitlist and seat them when tables become available.

File I/O: To ensure that your system can recover from crashes or restarts without losing data, you'll implement file input/output operations. This will involve reading from and writing to the layout and guest information files.

Files and Data Formats

Your system will use two main files to store data:

1. **Restaurant Layout (`layout.txt`):** This file will store information about each table in the restaurant, including the table number, the number of seats, and the name of the party occupying the table (if any). The data will be stored in a plain text format, with each table's information on a new line and values separated by commas. For example:

```
1,3,null
2,2,null
3,6,null
4,2,null
5,4,null
```

In this example, the restaurant has five tables. Table 1 has 3 seats, Table 2 has 2 seats, and so on. Initially, all tables are unoccupied (`null`).

2. **Guest Waitlist (`guest_info.txt`):** This file will store information about each party on the waitlist, including their name, the number of people in the party, and their contact information. The data will be stored in a CSV (Comma-Separated Values) format, with each party's information on a new line. For example:

   ```
   Jake,3,8123123
   Martha,1,12312
   Sonya,4,312312
   ```

   In this example, there are three parties on the waitlist. The first party, "Jake," has 3 people and a contact number of 8123123.

Project Setup

The basic outline of this project has already been made for you. If you look at the contents of the `code` folder (see filetree to the left), you will see:

```
code
└── Reservation-Project
    ├── guest_info.txt
    ├── layout.txt
    └── Restaurant.cpp
```

The project itself resides in the `Reservation-Project` directory. We will be building out the content of these files as the project progresses. At the end, the `Reservation-Project` directory (and its contents) will be uploaded to your GitHub portfolio.

Remember to keep your code organized and well-commented, as this will help you understand and debug your program. Good luck, and let's get started!

Handling the Restaurant Layout

Layout

In this section, we'll focus on managing the restaurant's layout, which involves keeping track of the tables and their availability. We will store this information in a file called `layout.txt`, ensuring that the table information is preserved even if the program is not running.

The Layout File

The `layout.txt` file will store information about each table in the restaurant, including the table number, the number of seats at the table, and the name of the party occupying the table (if any). The data will be stored in a plain text format, with each table's information on a new line and values separated by commas. For example:

```
1,3,null
2,2,null
3,6,null
4,2,null
5,4,null
```

In this layout, we have five tables. Table 1 has 3 seats, Table 2 has 2 seats, and so on. Initially, all tables are unoccupied (`null`).

Helper Methods for Managing the Layout

To interact with the layout file, we'll need to create some helper methods in our `Restaurant.cpp` file:

Reading the Layout: A method to read the layout from the `layout.txt` file and convert it into a C++ `std::map` object.

Updating the Layout: A method to update the layout (when a party is seated or a table is cleared) and save the changes back to the `layout.txt` file.

Before we get to the helper methods, we first need to include any required headers and define the `Restaurant` class. We need headers for file reading and writing, handling exceptions, and working with maps. Add the code below to the editor.

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <map>
#include <vector>
#include <stdexcept>

class Restaurant {

};
```

Then, create a constant for the layout file. The `readLayout` method is a public method that returns a map that has `strings` for keys and a vector of strings for the value. The method does not have any parameters. Start the method by creating a map. In C++, you need to handle any exceptions when working with files. Create a try block and read the `layout.txt` file into an `std::ifstream` object. In the catch block, print an error message. Finally, return the map that was created earlier.

```
    const std::string LAYOUT_FILE = "code/Reservation-
        Project/layout.txt";

public:
    std::map<std::string, std::vector<std::string>> readLayout()
        {
        std::map<std::string, std::vector<std::string>> layout;
        try {
            std::ifstream reader(LAYOUT_FILE);
            if (!reader) {
                throw std::runtime_error("File not found.");
            }



        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
        return layout;
    }
```

▼ File path names

> The file path given for the `layout.txt` file is designed to work with Codio. If you were to run the project on your local machine, you would most likely need to update the file path.

The logic for reading the file and placing the information into the map resides in the try block. Create a string variable that will hold each line read from the file. As long as you have not reached the end of the file, split the line of text at each comma, and store the result in a vector of strings.

Take the first element from the aforementioned vector and use it as the key in the map. Take the next two elements and store them as a vector of strings for the value in the map.

```
            std::string line;
            while (std::getline(reader, line)) {
                std::istringstream ss(line);
                std::string part;
                std::vector<std::string> parts;
                while (std::getline(ss, part, ',')) {
                    parts.push_back(part);
                }
                layout[parts[0]] = {parts[1], parts[2]};
            }
```

The `updateLayout` method is a public helper method that takes a map and writes its contents to the `layout.txt` file. This way we can have persistent storage of changes to the reservation system. The method takes a map. Again, we need to handle IO exceptions since we are working with a file. Create a try block that instantiates an `std::ofstream` object linked to the `layout.txt` file. In the catch block, print an error message.

```
public:
    void updateLayout(const std::map<std::string,
        std::vector<std::string>>& layout) {
        try {
            std::ofstream writer(LAYOUT_FILE);
            if (!writer) {
                throw std::runtime_error("File not found.");
            }



        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
    }
```

Inside the try block, iterate over the `map`. The auto keyword can be used to iterate over the map entries. Extract the key, the first element from the value, and the second element from the value. Write them to the file with a comma (no spaces) separating them. Do not forget the newline character at the end. This will overwrite the contents of the `layout.txt` file with the contents found in the map.

```
            for (const auto& entry : layout) {
                writer << entry.first << "," << entry.second[0]
        << "," << entry.second[1] << "\n";
            }
```

Testing the Code

We are now ready to test these helper methods. Start by creating the main method for the class. We are going to test the `updateLayout` method first, which is the ability to write to the `layout.txt` file. Create the `writeTest` map and put some test data into the structure. Then call `updateLayout` to write this data to the `layout.txt` file. This checks that we are properly writing to the `layout.txt` file.

For example:

```
int main() {
    Restaurant r;

    std::map<std::string, std::vector<std::string>> writeTest;
    writeTest["1"] = {"3", "null"};
    writeTest["2"] = {"2", "null"};
    writeTest["3"] = {"6", "null"};
    writeTest["4"] = {"2", "null"};
    writeTest["5"] = {"4", "null"};

    r.updateLayout(writeTest);

    return 0;
};
```

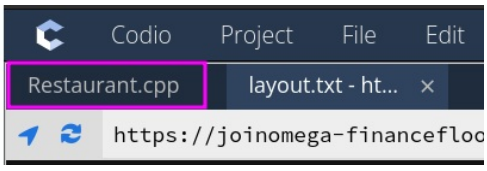
Running the code above will not produce any output; there are no print statements in the main method. After running the program, click the link to preview the `layout.txt` file.

You should see the following output:

```
1,3,null
2,2,null
3,6,null
4,2,null
5,4,null
```

▼ **Returning to the code editor**

> When previewing a text file, Codio will open a second tab in the panel with the file's contents. Click on the tab labeled `Restaurant.cpp` to return to the code editor.



Next, we need to test the ability to read the layout file. Modify the main method by adding a new map called `readTest` and set its value to the result of the `readLayout` method. Iterate over the `readTest` map, printing the key and values for each entry. This checks that we are properly reading from the `layout.txt` file.

```
    std::map<std::string, std::vector<std::string>> readTest =
        r.readLayout();
    for (const auto& entry : readTest) {
        std::cout << "Key: " << entry.first << ", Value: [" <<
        entry.second[0]
                  << ", " << entry.second[1] << "]" <<
        std::endl;
    }
```

▼ **Code**

> Your code should look like this:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <map>
#include <vector>
#include <stdexcept>

class Restaurant {
    const std::string LAYOUT_FILE = "code/Reservation-
        Project/layout.txt";

public:
    std::map<std::string, std::vector<std::string>>
        readLayout() {
        std::map<std::string, std::vector<std::string>>
        layout;
        try {
            std::ifstream reader(LAYOUT_FILE);
            if (!reader) {
                throw std::runtime_error("File not
        found.");
            }
            std::string line;
            while (std::getline(reader, line)) {
                std::istringstream ss(line);
                std::string part;
                std::vector<std::string> parts;
                while (std::getline(ss, part, ',')) {
                    parts.push_back(part);
                }
                layout[parts[0]] = {parts[1], parts[2]};
            }
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
        return layout;
    }
```

```
public:
    void updateLayout(const std::map<std::string,
        std::vector<std::string>>& layout) {
        try {
            std::ofstream writer(LAYOUT_FILE);
            if (!writer) {
                throw std::runtime_error("File not
        found.");
            }
            for (const auto& entry : layout) {
                writer << entry.first << "," <<
        entry.second[0] << "," << entry.second[1] << "\n";
            }
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
    }

};

int main() {
    Restaurant r;

    std::map<std::string, std::vector<std::string>>
        writeTest;
    writeTest["1"] = {"3", "null"};
    writeTest["2"] = {"2", "null"};
    writeTest["3"] = {"6", "null"};
    writeTest["4"] = {"2", "null"};
    writeTest["5"] = {"4", "null"};

    r.updateLayout(writeTest);

    std::map<std::string, std::vector<std::string>>
        readTest = r.readLayout();
    for (const auto& entry : readTest) {
        std::cout << "Key: " << entry.first << ", Value: ["
        << entry.second[0]
                  << ", " << entry.second[1] << "]" <<
        std::endl;
    }

    return 0;
};
```

Run the code above, and you should see the following output:

You should see the following output:

```
Key: 1, Value: [3, null]
Key: 2, Value: [2, null]
Key: 3, Value: [6, null]
Key: 4, Value: [2, null]
Key: 5, Value: [4, null]
```

This code sets up a default layout with five tables and then saves it to the `layout.txt` file. You can modify this layout to match the setup of your restaurant.

In the next section, we'll move on to managing the guest information and waitlist.

Handling Guest Information

Guest Information

In this section, we'll focus on handling guest information in our restaurant management system. We will store guest details in a file called `guest_info.txt`, which will help us keep track of the guests on the waitlist.

The Guest Info File

The `guest_info.txt` file will store information about each party waiting to be seated. Each line in the file will represent a party, with the party's name, the number of people in the party, and their contact information separated by commas. For example:

```
Jake,3,8123123
Martha,1,12312
Sonya,4,312312
```

In this example, the first line indicates that there is a party named `Jake` with `3` people and a contact number of `8123123`.

Helper Methods for Managing Guests

To interact with the guest information file, we'll create some helper methods in our `Restaurant.cpp` file:

Adding a Party to the Waitlist: A method to add a new party to the `guest_info.txt` file.

Reading the Waitlist: A method to read the waitlist from the `guest_info.txt` file and return it as a list.

First, we need to create a constant for the `guest_info.txt` file. We are going to add that just below the constant for the layout file.

```
    const std::string GUEST_FILE = "code/Reservation-
        Project/guest_info.txt";
```

▼ **File path names**

> The file path given for the `guest_info.txt` file is designed to work with Codio. If you were to run the project on your local machine, you would most likely need to update the file path.

Next, we are going to the `addPartyToWaitlist` method. This public method takes strings representing the name of the party, the size of the party, and a contact number for the party. The method does not return a value. Just as before we need to set up try and catch blocks to handle IO exceptions. In the try block, create an `ofstream` object linked to `GUEST_FILE`. Then write the party name, size, and contact information to the file. Use a comma (no spaces) to separate the information. In the case of an IO exception, print an error message.

```
void addPartyToWaitlist(const std::string &name, int size,
    const std::string &contact) {
    try {
        std::ofstream writer(GUEST_FILE, std::ios::app);
        if (writer.is_open()) {
            writer << name << "," << size << "," << contact
<< "\n";
            writer.close();
        } else {
            throw std::ios_base::failure("Error opening
file");
        }
    } catch (const std::exception &e) {
        std::cerr << e.what() << std::endl;
    }
}
```

▼ Did you notice?

> When creating the `ofstream` object for the `addPartyToWaitlist` method, we passed in the parameter `std::ios::app`. This means we are writing to the file in append mode. Unlike the `updateLayout` method which overwrites the file each time, writing in append mode will add new information at the end of the existing file. It does not alter any existing text.
>
> That means if you run the methods on this page more than once without changing the code, you will add duplicate information to the file. If you find yourself wanting to erase information from the `guest_info.txt` file, see the “Code” dropdown toward the bottom of the page. It has instructions on how to do so.

The public method `readWaitlist` reads from the file and returns a vector of string vectors. Each string vector contains the information for each group on the waitlist. Create the variable `waitlist`. Then, set up try and catch blocks that create an `ifstream` object linked to `GUEST_FILE` and print an error message for an IO exception. Don’t forget to return `waitlist` at the end of the method.

```
    std::vector<std::vector<std::string>> readWaitlist() {
        std::vector<std::vector<std::string>> waitlist;
        try {
            std::ifstream reader(GUEST_FILE);
            if (!reader.is_open()) {
                throw std::ios_base::failure("Error opening
        file");
            }
            std::string line;
            while (std::getline(reader, line)) {
                std::stringstream ss(line);
                std::vector<std::string> partyDetails;
                std::string detail;
                while (std::getline(ss, detail, ',')) {
                    partyDetails.push_back(detail);
                }
                waitlist.push_back(partyDetails);
            }
            reader.close();
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
        return waitlist;
    }
```

Testing the Code

We are going to test the new helper methods by rewriting the `main` method. We are first going to test the ability to write to the `guest_info.txt` file. Add three different groups to the waitlist using the `addPartyToWaitlist` method. Next, we need to test if we are properly reading the `guest_info.txt` file. Create the variable `waitlist` as a vector of string vectors. Set its value to the result of the `readWaitlist` method. Then iterate over `waitlist` and print the elements in the string vector.

```
int main() {
    Restaurant r;

    // Add some parties to the waitlist
    r.addPartyToWaitlist("Jake", 3, "8123123");
    r.addPartyToWaitlist("Martha", 1, "12312");
    r.addPartyToWaitlist("Sonya", 4, "312312");

    // Read and print the current waitlist
    std::vector<std::vector<std::string>> waitlist =
        r.readWaitlist();
    for (const auto &party : waitlist) {
        std::cout << "Party Name: " << party[0] << ", Size: " <<
        party[1] << ", Contact: " << party[2] << std::endl;
    }
    return 0;
};
```

▼ **Code**

> Your code should look like this:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <map>
#include <vector>
#include <stdexcept>

class Restaurant {
    const std::string LAYOUT_FILE = "code/Reservation-
        Project/layout.txt";
    const std::string GUEST_FILE = "code/Reservation-
        Project/guest_info.txt";

public:
    std::map<std::string, std::vector<std::string>>
        readLayout() {
        std::map<std::string, std::vector<std::string>>
        layout;
        try {
            std::ifstream reader(LAYOUT_FILE);
            if (!reader) {
                throw std::runtime_error("File not
        found.");
            }
            std::string line;
            while (std::getline(reader, line)) {
                std::istringstream ss(line);
                std::string part;
                std::vector<std::string> parts;
                while (std::getline(ss, part, ',')) {
                    parts.push_back(part);
                }
                layout[parts[0]] = {parts[1], parts[2]};
            }
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
        return layout;
    }

    void updateLayout(const std::map<std::string,
        std::vector<std::string>>& layout) {
        try {
            std::ofstream writer(LAYOUT_FILE);
            if (!writer) {
                throw std::runtime_error("File not
        found.");
            }
            for (const auto& entry : layout) {
                writer << entry.first << "," <<
        entry.second[0] << "," << entry.second[1] << "\n";
            }
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
```

```
    }

    void addPartyToWaitlist(const std::string &name, int
        size, const std::string &contact) {
        try {
            std::ofstream writer(GUEST_FILE,
        std::ios::app);
            if (writer.is_open()) {
                writer << name << "," << size << "," <<
        contact << "\n";
                writer.close();
            } else {
                throw std::ios_base::failure("Error opening
        file");
            }
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
    }

    std::vector<std::vector<std::string>> readWaitlist() {
        std::vector<std::vector<std::string>> waitlist;
        try {
            std::ifstream reader(GUEST_FILE);
            if (!reader.is_open()) {
                throw std::ios_base::failure("Error opening
        file");
            }
            std::string line;
            while (std::getline(reader, line)) {
                std::stringstream ss(line);
                std::vector<std::string> partyDetails;
                std::string detail;
                while (std::getline(ss, detail, ',')) {
                    partyDetails.push_back(detail);
                }
                waitlist.push_back(partyDetails);
            }
            reader.close();
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
        return waitlist;
    }

};


int main() {
    Restaurant r;

    // Add some parties to the waitlist
    r.addPartyToWaitlist("Jake", 3, "8123123");
    r.addPartyToWaitlist("Martha", 1, "12312");
    r.addPartyToWaitlist("Sonya", 4, "312312");

    // Read and print the current waitlist
    std::vector<std::vector<std::string>> waitlist =
        r.readWaitlist();
```

```
    for (const auto &party : waitlist) {
        std::cout << "Party Name: " << party[0] << ", Size:
        " << party[1] << ", Contact: " << party[2] <<
        std::endl;
    }
    return 0;
};
```

If you have run the program more than one time, you might have noticed that your `guest_info.txt` file is accumulating duplicate information. That is because we are writing to this file in append mode, which means new text (even duplicate text) gets added to the end of the file. If you want to manually clean up the file, click the link below. You can edit the file to match the expected output from this page. Codio will automatically save any changes you make.

Running the code above will produce visible output as well as write to a file. You should see the following output:

```
Party Name: Jake, Size: 3, Contact: 8123123
Party Name: Martha, Size: 1, Contact: 12312
Party Name: Sonya, Size: 4, Contact: 312312
```

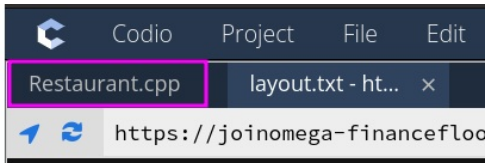
Click on the link below to open a preview of the `guest_info.txt` file.

You should see the following output:

```
Jake,3,8123123
Martha,1,12312
Sonya,4,312312
```

▼ Returning to the code editor

When previewing a text file, Codio will open a second tab in the panel with the file's contents. Click on the tab labeled `Restaurant.cpp` to return to the code editor.



In the next section, we'll integrate these components into our restaurant management system and implement the logic for seating parties and managing the layout.

Seating Logic

Finding a Table

With the basic file operations in place for managing the restaurant layout and guest waitlist, it's time to integrate these components and implement the logic for seating parties.

Seating a Party

When a party is ready to be seated, we need to check if there is a suitable table available. If there is, we will update the layout to indicate that the table is occupied and remove the party from the waitlist. Here's how we can implement this logic:

1. **Find a Suitable Table:** Look for a table that can accommodate the party size and is currently unoccupied.
2. **Update the Layout:** If a suitable table is found, update the layout to mark the table as occupied by the party.
3. **Remove the Party from the Waitlist:** Once the party is seated, remove their information from the `guest_info.txt` file.

Start by creating the `seatParty` method. It does not have any parameters, nor does it return a value. Create a vector of string arrays and set its value to the result of the `readWaitlist` method. Then create a map and set its value to the result of the `readLayout` method. If the waitlist is empty, print a message saying this and exit the method.

```
    void seatParty() {
        std::vector<std::vector<std::string>> waitlist =
        readWaitlist();
        std::map<std::string, std::vector<std::string>> layout =
        readLayout();

        if (waitlist.empty()) {
            std::cout << "The waitlist is empty." << std::endl;
            return;
        }
    }
```

If there are people on the waitlist, loop over it with an iterator. Extract a vector of strings for each party on the list. Then extract the party name and party size from the vector. Be sure to parse the party size as an integer. If the loop ends without exiting the method, then print a message that there is no suitable table for anyone on the waitlist.

```
        auto iterator = waitlist.begin();
        while (iterator != waitlist.end()) {
            std::vector<std::string> party = *iterator;
            std::string partyName = party[0];
            int partySize = std::stoi(party[1]);
```

After we have the party name and size, iterate over the tables, extracting the table information for each table. Then extract the capacity of the table and who is occupying the table.

```
            for (auto& table : layout) {
                std::vector<std::string> tableDetails =
        table.second;
                int tableSize = std::stoi(tableDetails[0]);
                std::string tableOccupant = tableDetails[1];
```

If the table is large enough to accommodate the party and nobody is currently using the table, look up the table in the layout map and set its value to a vector that has the table size and party name now sitting at the table. Be sure to convert the table size to a string. After seating people at the table, we need to remove the party from the waitlist. Then write the new waitlist to the file with the `updateWaitlist` method. Finally, print a message about the new party sitting at the table and exit the method.

```
                if (tableSize >= partySize && tableOccupant ==
        "null") {
                    // Seat the party at this table
                    layout[table.first] =
        {std::to_string(tableSize), partyName};
                    updateLayout(layout);

                    // Remove the party from the waitlist
                    iterator = waitlist.erase(iterator);
                    updateWaitlist(waitlist);

                    std::cout << "Seated party " << partyName <<
        " at table " << table.first << std::endl;
                    return;
                }
            }
            iterator++;
        }

        std::cout << "No suitable table available for any party
        on the waitlist." << std::endl;
```

In the previous method, we called the `updateWaitlist` method, which does not yet exist. This method takes a vector of string arrays representing the new waitlist and writes it to the `guest_info.txt` file. Set up a try block that creates an `ofstream` object linked to the `guest_info.txt` file. Iterate over the

waitlist, and write each element in the vector to the file. Use commas (no spaces) to separate the elements. We also need a newline character at the end. In the catch block, print the error message for any exceptions.

```
    void updateWaitlist(const
        std::vector<std::vector<std::string>>& waitlist) {
        try {
            std::ofstream writer(GUEST_FILE);
            for (const auto& party : waitlist) {
                writer << party[0] << "," << party[1] << "," <<
        party[2] << "\n";
            }
            writer.close();
        } catch (const std::exception& e) {
            std::cerr << e.what() << std::endl;
        }
    }
```

Testing the Code

With the seating logic in place, you can now test the system by updating the `main` method. The waitlist should now be empty. Start by seating the three parties already in the guest file. Then seat a nonexistent party. Add parties for Alice and Bob to the wait list and then seat them. Finally, get the layout of the restaurant and print the status of the tables.

```
int main() {
    Restaurant r;

    // Seat existing parties on the waitlist
    r.seatParty();
    r.seatParty();
    r.seatParty();

    // Seat non-existent party
    std::cout << std::endl;
    r.seatParty();

    // Add parties to the waitlist
    r.addPartyToWaitlist("Alice", 2, "123456");
    r.addPartyToWaitlist("Bob", 4, "654321");

    // Try to seat parties
    std::cout << std::endl;
    r.seatParty();
    r.seatParty();

    // Print the updated layout
    std::cout << std::endl;
    std::map<std::string, std::vector<std::string>> layout =
        r.readLayout();
    for (const auto& table : layout) {
        std::vector<std::string> tableDetails = table.second;
        std::cout << "Table " << table.first << ": Seats " <<
        tableDetails[0] << ", Occupied by " << tableDetails[1]
        << std::endl;
    }

    return 0;
};
```

▼ Code

> If you have run the program more than one time, you might have noticed that your `guest_info.txt` file is accumulating duplicate information. That is because we are writing to this file in append mode, which means new text (even duplicate text) gets added to the end of the file. If you want to manually clean up the file, click the link below. You can edit the file to match the expected output from this page.
>
> Your code should look like this:
>
> ```
> #include <iostream>
> #include <fstream>
> #include <sstream>
> #include <map>
> #include <vector>
> #include <stdexcept>
>
> class Restaurant {
> ```

```
    const std::string LAYOUT_FILE = "code/Reservation-
        Project/layout.txt";
    const std::string GUEST_FILE = "code/Reservation-
        Project/guest_info.txt";

public:
    std::map<std::string, std::vector<std::string>>
        readLayout() {
        std::map<std::string, std::vector<std::string>>
        layout;
        try {
            std::ifstream reader(LAYOUT_FILE);
            if (!reader) {
                throw std::runtime_error("File not
        found.");
            }
            std::string line;
            while (std::getline(reader, line)) {
                std::istringstream ss(line);
                std::string part;
                std::vector<std::string> parts;
                while (std::getline(ss, part, ',')) {
                    parts.push_back(part);
                }
                layout[parts[0]] = {parts[1], parts[2]};
            }
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
        return layout;
    }

    void updateLayout(const std::map<std::string,
        std::vector<std::string>>& layout) {
        try {
            std::ofstream writer(LAYOUT_FILE);
            if (!writer) {
                throw std::runtime_error("File not
        found.");
            }
            for (const auto& entry : layout) {
                writer << entry.first << "," <<
        entry.second[0] << "," << entry.second[1] << "\n";
            }
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
    }

    void addPartyToWaitlist(const std::string &name, int
        size, const std::string &contact) {
        try {
            std::ofstream writer(GUEST_FILE,
        std::ios::app);
            if (writer.is_open()) {
                writer << name << "," << size << "," <<
        contact << "\n";
                writer.close();
            } else {
```

```
            throw std::ios_base::failure("Error opening
    file");
        }
    } catch (const std::exception &e) {
        std::cerr << e.what() << std::endl;
    }
}

std::vector<std::vector<std::string>> readWaitlist() {
    std::vector<std::vector<std::string>> waitlist;
    try {
        std::ifstream reader(GUEST_FILE);
        if (!reader.is_open()) {
            throw std::ios_base::failure("Error opening
    file");
        }
        std::string line;
        while (std::getline(reader, line)) {
            std::stringstream ss(line);
            std::vector<std::string> partyDetails;
            std::string detail;
            while (std::getline(ss, detail, ',')) {
                partyDetails.push_back(detail);
            }
            waitlist.push_back(partyDetails);
        }
        reader.close();
    } catch (const std::exception &e) {
        std::cerr << e.what() << std::endl;
    }
    return waitlist;
}

void seatParty() {
    std::vector<std::vector<std::string>> waitlist =
    readWaitlist();
    std::map<std::string, std::vector<std::string>>
    layout = readLayout();

    if (waitlist.empty()) {
        std::cout << "The waitlist is empty." <<
    std::endl;
        return;
    }

    auto iterator = waitlist.begin();
    while (iterator != waitlist.end()) {
        std::vector<std::string> party = *iterator;
        std::string partyName = party[0];
        int partySize = std::stoi(party[1]);

        for (auto& table : layout) {
            std::vector<std::string> tableDetails =
    table.second;
            int tableSize = std::stoi(tableDetails[0]);
            std::string tableOccupant =
    tableDetails[1];
            if (tableSize >= partySize && tableOccupant
    == "null") {
```

```
                    // Seat the party at this table
                    layout[table.first] =
        {std::to_string(tableSize), partyName};
                    updateLayout(layout);

                    // Remove the party from the waitlist
                    iterator = waitlist.erase(iterator);
                    updateWaitlist(waitlist);

                    std::cout << "Seated party " <<
        partyName << " at table " << table.first <<
        std::endl;
                    return;
                }
            }
            iterator++;
        }

        std::cout << "No suitable table available for any
        party on the waitlist." << std::endl;
    }

    void updateWaitlist(const
        std::vector<std::vector<std::string>>& waitlist) {
        try {
            std::ofstream writer(GUEST_FILE);
            for (const auto& party : waitlist) {
                writer << party[0] << "," << party[1] <<
        "," << party[2] << "\n";
            }
            writer.close();
        } catch (const std::exception& e) {
            std::cerr << e.what() << std::endl;
        }
    }

};

int main() {
    Restaurant r;

    // Seat existing parties on the waitlist
    r.seatParty();
    r.seatParty();
    r.seatParty();

    // Seat non-existent party
    std::cout << std::endl;
    r.seatParty();

    // Add parties to the waitlist
    r.addPartyToWaitlist("Alice", 2, "123456");
    r.addPartyToWaitlist("Bob", 4, "654321");

    // Try to seat parties
    std::cout << std::endl;
    r.seatParty();
    r.seatParty();
```

```
    // Print the updated layout
    std::cout << std::endl;
    std::map<std::string, std::vector<std::string>> layout
        = r.readLayout();
    for (const auto& table : layout) {
        std::vector<std::string> tableDetails =
        table.second;
        std::cout << "Table " << table.first << ": Seats "
        << tableDetails[0] << ", Occupied by " <<
        tableDetails[1] << std::endl;
    }

    return 0;
};
```

You should see the following output:

```
Seated party Jake at table 1
Seated party Martha at table 2
Seated party Sonya at table 3

The waitlist is empty.

Seated party Alice at table 4
Seated party Bob at table 5

Table 1: Seats 3, Occupied by Jake
Table 2: Seats 2, Occupied by Martha
Table 3: Seats 6, Occupied by Sonya
Table 4: Seats 2, Occupied by Alice
Table 5: Seats 4, Occupied by Bob
```

In the next section, we will add functionality to clear tables when parties leave and to view the current waitlist and occupied tables.

Clearing Tables and Viewing Information

Expanding Functionality

Now that we have implemented the core functionality of seating parties, let's expand our system to include the ability to clear tables when parties leave and to view the current waitlist and occupied tables.

Clearing a Table

When a party leaves a table, we need to update the layout to mark the table as unoccupied. Here's how we can implement this functionality:

1. **Update the Layout:** Change the table's status to unoccupied in the layout.
2. **Save the Updated Layout:** Write the updated layout back to the `layout.txt` file.

Create the `clearTable` method which takes a string representing a table number. The method does not return a value. Fetch the current layout with the `readLayout` method. If the method contains the current table number, extract the details for the given table. Set the second element to `null` to signify the table is unoccupied. Then put the new table details into the layout. Call `updateLayout` to write the new layout to the file. Print a message that the table has been cleared. If the given table number is not in the layout, print a message that the table does not exist.

```
    void clearTable(const std::string &tableNumber) {
        auto layout = readLayout();

        if (layout.find(tableNumber) != layout.end()) {
            auto& tableDetails = layout[tableNumber];
            tableDetails[1] = "null"; // Mark the table as
        unoccupied
            updateLayout(layout);

            std::cout << "Table " << tableNumber << " has been
        cleared." << std::endl;
        } else {
            std::cout << "Table " << tableNumber << " does not
        exist." << std::endl;
        }
    }
```

Viewing the Waitlist and Occupied Tables

It's also useful to be able to view the current waitlist and a list of occupied tables. We can implement this with the following functionality:

1. **View Waitlist:** Returns a list of parties on the waitlist.
2. **View Occupied Tables:** Returns a list of tables that are currently occupied.

The `viewWaitlist` method does not have any parameters, nor does it return a value. Start by extracting the waitlist with the `readWaitlist` method. Then create a new list of strings. Iterate over the waitlist and add each element from the string array to the list with some text that explains the text stored in the file. Return the new list that contains the waitlist information.

```
    std::vector<std::string> viewWaitlist() {
        auto waitlist = readWaitlist();
        std::vector<std::string> waitlistInfo;

        for (const auto& party : waitlist) {
            waitlistInfo.push_back("Party Name: " + party[0] +
        ", Size: " + party[1] + ", Contact: " + party[2]);
        }

        return waitlistInfo;
    }
```

The `viewOccupiedTables` method does not have any parameters and returns a list of strings. Start by extracting the layout with the `readLayout` method. Then create a new list of strings representing the occupied tables. Iterate over the entries in the layout map. Extract the table number and the details for the tables. If the table is occupied, add the table number and party name to the occupied tables list. Don't forget to include some text to provide meaning for this information. Finally, return the occupied tables list.

```
    std::vector<std::string> viewOccupiedTables() {
        auto layout = readLayout();
        std::vector<std::string> occupiedTables;

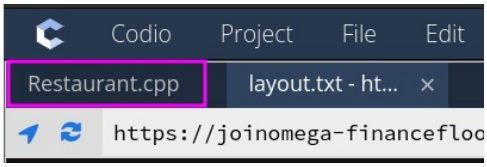
        for (const auto& entry : layout) {
            const std::string& tableNumber = entry.first;
            const auto& tableDetails = entry.second;
            if (tableDetails[1] != "null") {
                occupiedTables.push_back("Table " + tableNumber
        + ": Occupied by " + tableDetails[1]);
            }
        }

        return occupiedTables;
    }
```

Testing the Code

Before we begin testing, we need to clear out the `guest_info.txt` file. Use the link below to open the file. Erase all of the contents. Codio will automatically save these changes. Then return to the C++ file.

▼ **Returning to the code editor**

> When previewing a text file, Codio will open a second tab in the panel with the file's contents. Click on the tab labeled `Restaurant.cpp` to return to the code editor.
>
> 
>

You can now test the full functionality of your restaurant management system by adding, seating, and clearing parties, as well as viewing the waitlist and occupied tables. Here's an example of how you might use all these features together. Update the main method to be the following:

```
int main() {
    Restaurant r;

    // Add parties to the waitlist
    r.addPartyToWaitlist("Juanita", 3, "246810");
    r.addPartyToWaitlist("George", 5, "121416");

    // View the occupied tables
    std::cout << "Occupied Tables:" << std::endl;
    for (const auto& info : r.viewOccupiedTables()) {
        std::cout << info << std::endl;
    }

    // Clear a table
    std::cout << std::endl;
    r.clearTable("1");

    // Try to seat parties
    std::cout << std::endl;
    r.seatParty();
    r.seatParty();

    // View the updated waitlist and occupied tables
    std::cout << "\nUpdated Waitlist:" << std::endl;
    for (const auto& info : r.viewWaitlist()) {
        std::cout << info << std::endl;
    }

    std::cout << "\nUpdated Occupied Tables:" << std::endl;
    for (const auto& info : r.viewOccupiedTables()) {
        std::cout << info << std::endl;
    }

    return 0;
};
```

▼ **Code**

> If you have run the program more than one time, you might have noticed that your `guest_info.txt` file is accumulating duplicate information. That is because we are writing to this file in append mode, which means new text (even duplicate text) gets added to the end of the file. If you want to manually clean up the file, click the link below. You can remove all of the text in the file. Codio will automatically save any changes you make.
>
> Your code should look like this:
>
> ```
> #include <iostream>
> #include <fstream>
> #include <sstream>
> #include <map>
> #include <vector>
> #include <stdexcept>
> ```

```
class Restaurant {
    const std::string LAYOUT_FILE = "code/Reservation-
        Project/layout.txt";
    const std::string GUEST_FILE = "code/Reservation-
        Project/guest_info.txt";

public:
    std::map<std::string, std::vector<std::string>>
        readLayout() {
        std::map<std::string, std::vector<std::string>>
        layout;
        try {
            std::ifstream reader(LAYOUT_FILE);
            if (!reader) {
                throw std::runtime_error("File not
        found.");
            }
            std::string line;
            while (std::getline(reader, line)) {
                std::istringstream ss(line);
                std::string part;
                std::vector<std::string> parts;
                while (std::getline(ss, part, ',')) {
                    parts.push_back(part);
                }
                layout[parts[0]] = {parts[1], parts[2]};
            }
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
        return layout;
    }

    void updateLayout(const std::map<std::string,
        std::vector<std::string>>& layout) {
        try {
            std::ofstream writer(LAYOUT_FILE);
            if (!writer) {
                throw std::runtime_error("File not
        found.");
            }
            for (const auto& entry : layout) {
                writer << entry.first << "," <<
        entry.second[0] << "," << entry.second[1] << "\n";
            }
        } catch (const std::exception &e) {
            std::cerr << e.what() << std::endl;
        }
    }

    void addPartyToWaitlist(const std::string &name, int
        size, const std::string &contact) {
        try {
            std::ofstream writer(GUEST_FILE,
        std::ios::app);
            if (writer.is_open()) {
                writer << name << "," << size << "," <<
        contact << "\n";
                writer.close();
```

```
        } else {
            throw std::ios_base::failure("Error opening
    file");
        }
    } catch (const std::exception &e) {
        std::cerr << e.what() << std::endl;
    }
}

std::vector<std::vector<std::string>> readWaitlist() {
    std::vector<std::vector<std::string>> waitlist;
    try {
        std::ifstream reader(GUEST_FILE);
        if (!reader.is_open()) {
            throw std::ios_base::failure("Error opening
    file");
        }
        std::string line;
        while (std::getline(reader, line)) {
            std::stringstream ss(line);
            std::vector<std::string> partyDetails;
            std::string detail;
            while (std::getline(ss, detail, ',')) {
                partyDetails.push_back(detail);
            }
            waitlist.push_back(partyDetails);
        }
        reader.close();
    } catch (const std::exception &e) {
        std::cerr << e.what() << std::endl;
    }
    return waitlist;
}

void seatParty() {
    std::vector<std::vector<std::string>> waitlist =
    readWaitlist();
    std::map<std::string, std::vector<std::string>>
    layout = readLayout();

    if (waitlist.empty()) {
        std::cout << "The waitlist is empty." <<
    std::endl;
        return;
    }

    auto iterator = waitlist.begin();
    while (iterator != waitlist.end()) {
        std::vector<std::string> party = *iterator;
        std::string partyName = party[0];
        int partySize = std::stoi(party[1]);

        for (auto& table : layout) {
            std::vector<std::string> tableDetails =
    table.second;
            int tableSize = std::stoi(tableDetails[0]);
            std::string tableOccupant =
    tableDetails[1];
```

```
            if (tableSize >= partySize && tableOccupant
    == "null") {
                // Seat the party at this table
                layout[table.first] =
    {std::to_string(tableSize), partyName};
                updateLayout(layout);

                // Remove the party from the waitlist
                iterator = waitlist.erase(iterator);
                updateWaitlist(waitlist);

                std::cout << "Seated party " <<
    partyName << " at table " << table.first <<
    std::endl;
                return;
            }
        }
        iterator++;
    }

    std::cout << "No suitable table available for any
    party on the waitlist." << std::endl;
}

void updateWaitlist(const
    std::vector<std::vector<std::string>>& waitlist) {
    try {
        std::ofstream writer(GUEST_FILE);
        for (const auto& party : waitlist) {
            writer << party[0] << "," << party[1] <<
    "," << party[2] << "\n";
        }
        writer.close();
    } catch (const std::exception& e) {
        std::cerr << e.what() << std::endl;
    }
}

void clearTable(const std::string &tableNumber) {
    auto layout = readLayout();

    if (layout.find(tableNumber) != layout.end()) {
        auto& tableDetails = layout[tableNumber];
        tableDetails[1] = "null"; // Mark the table as
    unoccupied
        updateLayout(layout);

        std::cout << "Table " << tableNumber << " has
    been cleared." << std::endl;
    } else {
        std::cout << "Table " << tableNumber << " does
    not exist." << std::endl;
    }
}

std::vector<std::string> viewWaitlist() {
    auto waitlist = readWaitlist();
    std::vector<std::string> waitlistInfo;

    for (const auto& party : waitlist) {
```

```
            waitlistInfo.push_back("Party Name: " +
        party[0] + ", Size: " + party[1] + ", Contact: " +
        party[2]);
        }

        return waitlistInfo;
    }

    std::vector<std::string> viewOccupiedTables() {
        auto layout = readLayout();
        std::vector<std::string> occupiedTables;

        for (const auto& entry : layout) {
            const std::string& tableNumber = entry.first;
            const auto& tableDetails = entry.second;
            if (tableDetails[1] != "null") {
                occupiedTables.push_back("Table " +
        tableNumber + ": Occupied by " + tableDetails[1]);
            }
        }

        return occupiedTables;
    }

};

int main() {
    Restaurant r;

    // Add parties to the waitlist
    r.addPartyToWaitlist("Juanita", 3, "246810");
    r.addPartyToWaitlist("George", 5, "121416");

    // View the occupied tables
    std::cout << "Occupied Tables:" << std::endl;
    for (const auto& info : r.viewOccupiedTables()) {
        std::cout << info << std::endl;
    }

    // Clear a table
    std::cout << std::endl;
    r.clearTable("1");

    // Try to seat parties
    std::cout << std::endl;
    r.seatParty();
    r.seatParty();

    // View the updated waitlist and occupied tables
    std::cout << "\nUpdated Waitlist:" << std::endl;
    for (const auto& info : r.viewWaitlist()) {
        std::cout << info << std::endl;
    }

    std::cout << "\nUpdated Occupied Tables:" << std::endl;
    for (const auto& info : r.viewOccupiedTables()) {
        std::cout << info << std::endl;
    }
```

```
    return 0;
};
```

You should see the following output:

```
Occupied Tables:
Table 1: Occupied by Jake
Table 2: Occupied by Martha
Table 3: Occupied by Sonya
Table 4: Occupied by Alice
Table 5: Occupied by Bob

Table 1 has been cleared.

Seated party Juanita at table 1
No suitable table available for any party on the waitlist.

Updated Waitlist:
Party Name: George, Size: 5, Contact: 121416

Updated Occupied Tables:
Table 1: Occupied by Juanita
Table 2: Occupied by Martha
Table 3: Occupied by Sonya
Table 4: Occupied by Alice
Table 5: Occupied by Bob
```

In the next section, we'll discuss how to further enhance and customize your system to suit the needs of a real restaurant.

Going Further

Enhancing and Customizing Your System

Now that you have a basic restaurant management system in place, you can consider enhancing and customizing it to better suit the needs of a real restaurant. Here are some ideas for further development:

Handling Table Merging and Splitting

In many restaurants, tables can be merged to accommodate larger parties or split to accommodate smaller parties. You can add functionality to your system to handle these scenarios, allowing for more flexible table management.

Implementing a Reservation System

In addition to managing walk-in guests, you can expand your system to handle reservations. This would involve adding functionality to book tables for specific times and dates and managing the reservation list alongside the walk-in waitlist.

Adding a Graphical User Interface (GUI)

While the current system uses a command-line interface, you can enhance the user experience by adding a graphical user interface (GUI). A GUI can provide a more intuitive and visually appealing way for restaurant staff to interact with the system.

Integrating with Other Systems

In a real restaurant, the table management system might need to integrate with other systems, such as point-of-sale (POS) systems or kitchen management systems. Consider how your system could interface with these other systems to provide a seamless experience.

Implementing Analytics and Reporting

Adding analytics and reporting features to your system can provide valuable insights for the restaurant. For example, you could track how long parties typically wait before being seated, which tables are most frequently used, or peak times for guest arrivals.

Handling Special Requests and Preferences

Guests may have special requests or preferences, such as a preference for a window seat or a need for a high chair. You can add functionality to your system to record and accommodate these special requests when seating parties.

Building a restaurant management system is a great way to apply data structure concepts in a real-world scenario. As you enhance and customize your system, you'll gain experience in problem-solving and software development that will be valuable in your future projects. Remember to test your system thoroughly and gather feedback from potential users to ensure that it meets the needs of a busy restaurant environment.

GitHub

Uploading Your Work to GitHub

GitHub is a popular platform for hosting and sharing code repositories. Uploading your work to GitHub not only provides a backup of your project but also allows you to showcase your work to potential employers or collaborators. Here's a step-by-step guide on how to upload your restaurant management system project to GitHub:

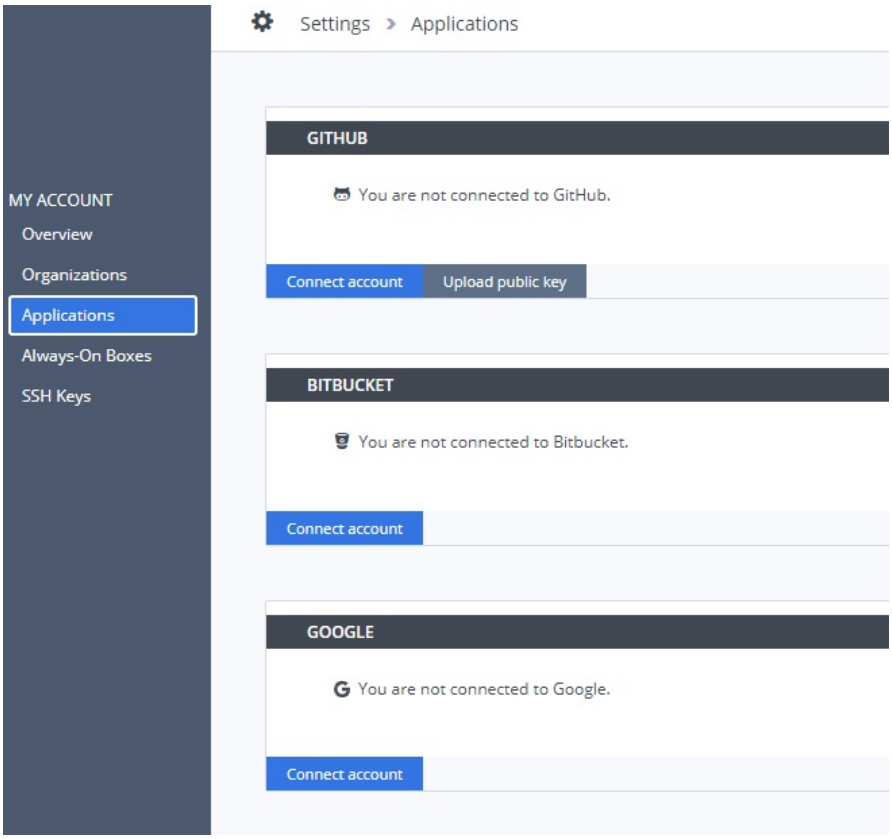
Create a GitHub Account

If you don't already have a GitHub account, go to GitHub and sign up for a free account. You'll need to provide a username, email address, and password.

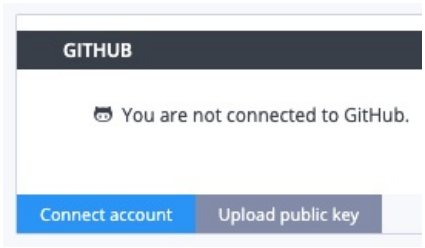
Connecting GitHub and Codio

You need to connect GitHub to your Codio account. This only needs to be done one time.

- In your Codio account, click on your username
- Click on **Applications**



- Under GitHub, click on **Connect account**



- You will be using an SSH connection, so you need to click on **Upload public key**

Install Git

Git is the version control system that GitHub is built on. Codio already has Git installed, but if you are working on your own machine, you will need to install this yourself. You can download Git from git-scm.com.

Configure Git

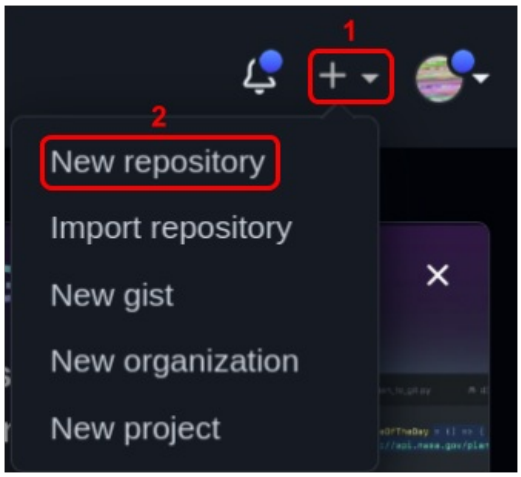
Open a terminal (left panel) or command prompt and configure your Git username and email using the following commands:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

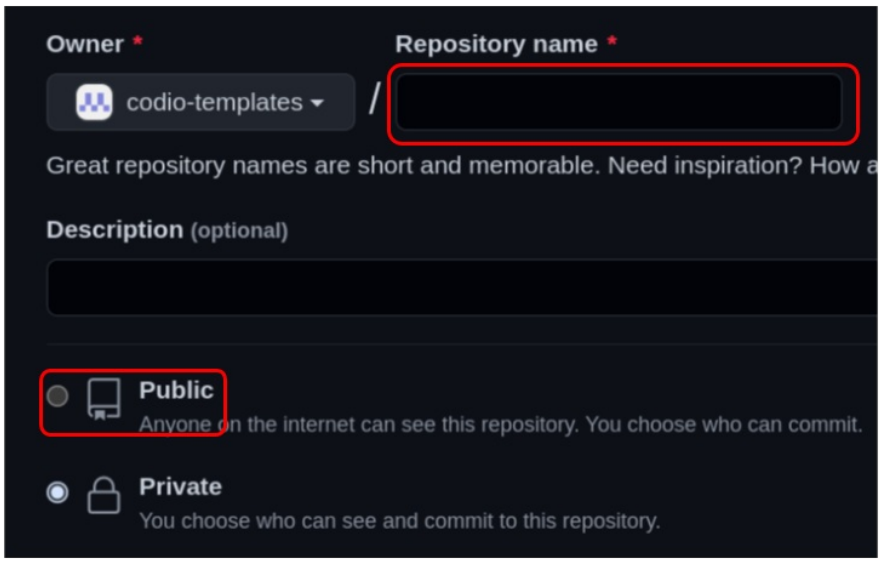
Replace “Your Name” and “your_email@example.com” with your actual name and email address.

Create a New Repository on GitHub

Go to the GitHub website and log in to your GitHub account. Click the “+” icon in the top-right corner and select “New repository.”



Give your repository a name (e.g., “restaurant-management-system”) and an optional description. Choose whether the repository will be public or private. We suggest making the repository public so that others can see and contribute to your work.



Click “Create repository” to finalize the repository creation process.

Setting Up a Local Git Repository

After your repository is complete, GitHub will provide an address for it. **Important**, make sure that you select the SSH address. You can tell if it is an SSH address because it should start with `git@github.com:`. Once you have selected the SSH address, click the button on the right to make a copy of it. We need this address to connect your GitHub repo to your reservation project.



Once you have copied the SSH address for your repository, return to Codio. In the command line, we are going to change to the `code` directory, which is where the reservation project resides. Then we are going to initialize a local Git repository.

```
cd code && git init
```

Next, we need to add a remote repository so that your local repository is linked to the repository on the website. Enter the following command in the terminal. Be sure to replace `<repository-ssh>` with the SSH address you copied from the GitHub website.

```
git remote add origin <repository-ssh>
```

Add, Commit, & Push

The standard Git workflow is add, commit, and push. The `add` command adds files to the staging area. The `commit` command copies the files from the staging area to the local repository. You should add a message to each commit explaining the changes made to the files. The final command is `push` which copies the files from the local repository to the remote repository (the GitHub website).

Start by adding your project files to the staging area using the `add` command. The dot (`.`) tells Git to add all files in the current directory.

```
git add .
```

Commit your changes to the local repository with a message describing what you're committing. Using "Initial commit" as your first commit is a common practice, but you can use any message that describes what changes were made.

```
git commit -m "Initial commit"
```

Finally, push your code to GitHub using the following command:

```
git push -u origin master
```

If your default branch is named `main` (as in newer repositories), use `git push -u origin main` instead. From here on out, you can use a simple `git push` as the command. The command above only needs to be run the first time you push.

Check Your Repository on GitHub

Go to your GitHub repository page, and you should see your project files there. Congratulations! You've successfully uploaded your project to GitHub.

By uploading your work to GitHub, you not only ensure that your code is safely stored in a remote repository, but you also make it accessible for collaboration and sharing with others in the developer community.